

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу
«Операционные системы»

Группа: М8О-210БВ-25

Студент: Хабиров Р.И.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 08.10.26

Москва, 2026

Постановка задачи

Вариант 9.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы.

Рассчитать детерминант матрицы (используя определение детерминанта).

Общий метод и алгоритм решения

В основе программы лежит метод Лапласа (разложение по строке). Определитель матрицы $n \cdot n$ находится через сумму произведений элементов первой строки на их алгебраические дополнения размера $(n - 1) \cdot (n - 1)$. Для первой строки матрицы формула имеет вид:

$$\det A = \sum_{j=0}^{n-1} (-1)^{i+j} \cdot a_{ij} \cdot M_{ij}$$

Также в данной лабораторной работе используются следующие метрики:

- **Ускорение:** $S = \frac{T_s}{T_p}$, где T_s – время последовательной реализации, T_p – время параллельной реализации, $1 \leq S \leq P$, p – количество ядер

Ускорение предназначается для того, чтобы показать, во сколько раз параллельная реализация программы работает быстрее, чем последовательная

- **Эффективность:** $X = \frac{s}{p}$, где $X < 1$

Эффективность показывает, насколько продуктивно используются выделенные потоки (какой процент их мощности тратится на полезные вычисления)

Алгоритм решения:

1. Программа считывает размер матрицы n из аргументов командной строки (`argv`) и генерирует матрицу. Из макроса `NUM_THREADS` считывается лимит одновременно работающих потоков.
2. В цикле по столбцам j (от 0 до $n-1$) для каждого элемента первой строки создаются независимые потоки через функцию `pthread_create`. Каждому потоку передается персональная структура данных `ThreadData`.
3. Как только количество активных потоков достигает заданного лимита, главный поток блокируется с помощью `pthread_join`. Он дожидается завершения текущей пачки потоков, суммирует их промежуточные результаты в общую переменную определителя, сбрасывает счетчик активных потоков и переходит к запуску следующей порции.
4. После выхода из основного цикла оставшиеся активные потоки (если размер матрицы n не делится нацело на лимит потоков) отдельно дожидаются через финальный вызов `pthread_join`, а их результаты добавляются к итоговой сумме.

5. Каждый созданный поток выделяет локальную память под один минор размером $(n-1) \times (n-1)$, заполняет его данными исходной матрицы (пропуская строку 0 и свой столбец j) и вызывает рекурсивный последовательный алгоритм вычисления детерминанта. После этого поток рассчитывает знак, умножает его на элемент матрицы, записывает результат в структуру и очищает память своего минора.
6. Точное время вычислений замеряется с помощью функции `clock_gettime`. Итоговый определитель и время работы программы выводятся на экран через системный вызов `write` напрямую в стандартный поток вывода.

Код программы

Программа `matrix.c`

```
#include <stdlib.h>

#include <unistd.h>

#include <time.h>

#include <pthread.h>

#include <math.h>

#ifndef NUM_THREADS
#define NUM_THREADS 1
#endif

typedef struct ThreadData{
    double **matrix;

    int row;

    int col;

    int n;

    int sign;

    double result;
} ThreadData;

double** create_matrix (int n){
    double **matrix = (double **)malloc(n * sizeof(double *));

    for (int i = 0; i < n; i++){
        matrix[i] = (double *)malloc(n * sizeof(double));
```

```

    }

    return matrix;
}

void clear(double **matrix, int n){
    for (int i = 0; i < n; i++){
        free(matrix[i]);
    }
    free(matrix);
}

double **get_minor(double **matrix, double **minor, int row, int col, int n){
    int i2 = 0;
    for (int i1 = 0; i1 < n; i1++){
        if (i1 == row){
            continue;
        }
        int j2 = 0;
        for (int j1 = 0; j1 < n; j1++){
            if (j1 == col){
                continue;
            }
            minor[i2][j2] = matrix[i1][j1];
            j2++;
        }
        i2++;
    }

    return minor;
}

```

```

double determinant(double **matrix, int n){
    double det = 0.0;
    int sign;
    if (n == 1){
        return matrix[0][0];
    }

    else if (n == 2){
        return matrix[0][0] * matrix[1][1] - (matrix[0][1] * matrix[1][0]);
    }

    else{
        double **minor = create_matrix(n - 1);
        int i = 0;
        for (int j = 0; j < n; j++){
            if ((i + j) % 2 == 0){
                sign = 1;
            }
            else{
                sign = -1;
            }
            det += sign * matrix[i][j] * determinant(get_minor(matrix, minor, i, j, n), n - 1);
        }
        clear(minor, n - 1);
        return det;
    }
}

```

```

void *parallel_det(void *arg){

```

```
ThreadData *data = (ThreadData *)arg;
```

```
double **minor = create_matrix(data->n - 1);
```

```
if ((data->row + data->col) % 2 == 0){
```

```
    data->sign = 1;
```

```
}
```

```
else{
```

```
    data->sign = -1;
```

```
}
```

```
    data->result = data->sign * data->matrix[data->row][data->col] * determinant(get_minor(data->matrix, minor, data->row, data->col, data->n), data->n - 1);
```

```
    clear(minor, data->n - 1);
```

```
    return NULL;
```

```
}
```

```
void print_str(char *str){
```

```
    int len = 0;
```

```
    while (str[len] != '\0'){
```

```
        len ++;
```

```
}
```

```
    write(1, str, len);
```

```
}
```

```
void print_double(double x){
```

```
    char buffer[64];
```

```
    int len_res = 0;
```

```
    int i = 0;
```

```

if (x < 0.0){
    buffer[i++] = '-';
}
x = fabs(x);
long long res = (long long) x;
int frac = (int)((x - res) * 100.0f + 0.5f);
if (frac >= 100) {
    res += 1;
    frac -= 100;
}
int len_frac = 2;

long long temp_res = res;
if (temp_res == 0){
    len_res = 1;
}
else{
    while (temp_res > 0){
        temp_res /= 10;
        len_res ++;
    }
}

while (len_res != 0){
    long long digit = res / pow(10, len_res - 1);
    buffer[i++] = (char)(digit) + '0';
    res -= digit * pow(10, len_res - 1);
    len_res -= 1;
}
buffer[i++] = '.';

```

```

while (len_frac != 0){
    int digit = frac / pow(10, len_frac - 1);
    buffer[i++] = (char)(digit) + '0';
    frac -= digit * pow(10, len_frac - 1);
    len_frac -= 1;
}

buffer[i] = '\0';

print_str(buffer);
}

int str_to_int(char *str){
    int res = 0;
    int i = 0;
    while (str[i] != '\0'){
        res = res * 10 + str[i] - '0';
        i++;
    }

    return res;
}

int main(int argc, char *argv[]){
    if (argc < 2){
        print_str("Использование ./a.out <размер_матрицы>\n");
        return 0;
    }

    pthread_t threads[NUM_THREADS];
    ThreadData args[NUM_THREADS];

```

```

int active_threads = 0;

double res = 0.0;

int i = 0;

int j = 0;

int n, elem;


n = str_to_int(argv[1]);

double **matrix = create_matrix(n);

for (int i = 0; i < n; i++){

    for (int j = 0; j < n; j++){

        matrix[i][j] = (rand() % 10) + 1;

    }

}


struct timespec start, end;

clock_gettime(CLOCK_MONOTONIC, &start);

while (j < n){

    if (active_threads == NUM_THREADS){

        for (int i = 0; i < NUM_THREADS; i++){

            pthread_join(threads[i], NULL);

            res += args[i].result;

        }

        active_threads = 0;

    }

    args[active_threads].matrix = matrix;

    args[active_threads].row = i;

    args[active_threads].col = j;

    args[active_threads].n = n;

    pthread_create(&threads[active_threads], NULL, parallel_det, &args[active_threads]);

    active_threads++;

```

```

        j++;
    };

    if (active_threads > 0){
        for (int i = 0; i < active_threads; i++){
            pthread_join(threads[i], NULL);
            res += args[i].result;
        }
        active_threads = 0;
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    double diff = (end.tv_sec - start.tv_sec) * 1000.0 + (end.tv_nsec - start.tv_nsec) / 1000000.0;
    print_str("Параллельный определитель равен: ");
    print_double(res);
    print_str(", время работы программы: ");
    print_double(diff);
    print_str(" мс\n");
}

```

Программа seq.c

```

#include <stdlib.h>

#include <unistd.h>

#include <time.h>

#include <math.h>

double** create_matrix(int n) {
    double **matrix = (double **)malloc(n * sizeof(double *));
    for (int i = 0; i < n; i++) {
        matrix[i] = (double *)malloc(n * sizeof(double));
    }
}

```

```
    }  
    return matrix;  
}
```

```
void clear(double **matrix, int n) {  
    for (int i = 0; i < n; i++) {  
        free(matrix[i]);  
    }  
    free(matrix);  
}
```

```
double** get_minor(double **matrix, double **minor, int row, int col, int n) {  
    int i2 = 0;  
    for (int i1 = 0; i1 < n; i1++) {  
        if (i1 == row){  
            continue;  
        }  
        int j2 = 0;  
        for (int j1 = 0; j1 < n; j1++) {  
            if (j1 == col){  
                continue;  
            }  
            minor[i2][j2] = matrix[i1][j1];  
            j2++;  
        }  
        i2++;  
    }  
    return minor;  
}
```

```

double determinant(double **matrix, int n) {
    if (n == 1){
        return matrix[0][0];
    }
    if (n == 2){
        return matrix[0][0] * matrix[1][1] - (matrix[0][1] * matrix[1][0]);
    }

    double det = 0.0;
    double **minor = create_matrix(n - 1);
    int i = 0;
    int sign;

    for (int j = 0; j < n; j++) {
        if ((i + j) % 2 == 0){
            sign = 1;
        }
        else{
            sign = -1;
        }

        det += sign * matrix[i][j] * determinant(get_minor(matrix, minor, i, j, n), n - 1);
    }
    clear(minor, n - 1);
    return det;
}

void print_str(char *str) {
    int len = 0;
    while (str[len] != '\0'){

```

```

        len++;
    }
    write(1, str, len);
}

```

```

void print_double(double x) {
    char buffer[64];
    int len_res = 0;
    int i = 0;
    if (x < 0.0) buffer[i++] = '-';
    x = fabs(x);
    long long res = (long long)x;
    int frac = (int)((x - res) * 100.0f + 0.5f);
    if (frac >= 100){
        res += 1;
        frac -= 100;
    }
    int len_frac = 2;

    long long temp_res = res;
    if (temp_res == 0){
        len_res = 1;
    }
    else {
        while (temp_res > 0){
            temp_res /= 10;
            len_res++;
        }
    }
    while (len_res != 0) {

```

```

    long long digit = res / (long long)pow(10, len_res - 1);

    buffer[i++] = (char)(digit) + '0';

    res -= digit * (long long)pow(10, len_res - 1);

    len_res--;

}

buffer[i++] = '.';

while (len_frac != 0) {

    int digit = frac / (int)pow(10, len_frac - 1);

    buffer[i++] = (char)(digit) + '0';

    frac -= digit * (int)pow(10, len_frac - 1);

    len_frac--;

}

buffer[i] = '\0';

print_str(buffer);

}

int str_to_int(char *str) {

    int res = 0;

    int i = 0;

    while (str[i] != '\0') {

        res = res * 10 + (str[i] - '0');

        i++;

    }

    return res;

}

int main(int argc, char *argv[]) {

    if (argc < 2) {

        print_str("Использование: ./a.out <размер_матрицы>\n");

        return 0;

    }

```

```

}

int n = str_to_int(argv[1]);

double **matrix = create_matrix(n);

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        matrix[i][j] = (rand() % 10) + 1;
    }
}

struct timespec start, end;

clock_gettime(CLOCK_MONOTONIC, &start);

double res = determinant(matrix, n);

clock_gettime(CLOCK_MONOTONIC, &end);

double diff = (end.tv_sec - start.tv_sec) * 1000.0 + (end.tv_nsec - start.tv_nsec) / 1000000.0;

print_str("Последовательный определитель равен: ");
print_double(res);
print_str(", время работы программы: ");
print_double(diff);
print_str(" мс\n");

clear(matrix, n);

return 0;
}

```

Протокол работы программы

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ gcc seq.c -lm
```

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ ./a.out 11
```

Последовательный определитель равен: 3301405825.00, время работы программы: 1502.62 мс

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ gcc matrix.c -lm -lpthread -DNUM_THREADS=1
```

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ ./a.out 11
```

Параллельный определитель равен: 3301405825.00, время работы программы: 1525.77 мс

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ gcc matrix.c -lm -lpthread -DNUM_THREADS=2
```

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ ./a.out 11
```

Параллельный определитель равен: 3301405825.00, время работы программы: 922.45 мс

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ gcc matrix.c -lm -lpthread -DNUM_THREADS=4
```

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ ./a.out 11
```

Параллельный определитель равен: 3301405825.00, время работы программы: 588.07 мс

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ gcc matrix.c -lm -lpthread -DNUM_THREADS=8
```

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ ./a.out 11
```

Параллельный определитель равен: 3301405825.00, время работы программы: 470.55 мс

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ gcc matrix.c -lm -lpthread -DNUM_THREADS=12
```

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ ./a.out 11
```

Параллельный определитель равен: 3301405825.00, время работы программы: 254.73 мс

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ gcc matrix.c -lm -lpthread -DNUM_THREADS=16
```

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ ./a.out 11
```

Параллельный определитель равен: 3301405825.00, время работы программы: 371.02 мс

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ gcc matrix.c -lm -lpthread -DNUM_THREADS=128
```

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ ./a.out 11
```

Параллельный определитель равен: 3301405825.00, время работы программы: 379.65 мс

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ gcc matrix.c -lm -lpthread -DNUM_THREADS=1024
```

```
flosy@Flosy:~/2_course/OS-labs/lab_2$ ./a.out 11
```

Параллельный определитель равен: 3301405825.00, время работы программы: 414.01 мс

Вывод

В данной лабораторной работе реализовывалось две версии алгоритма вычисления детерминанта матрицы по его определению, последовательный и параллельный. В ходе работы были использованы следующие функции и системные вызовы:

- `pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg)` - создание нового потока.
- `pthread_join(pthread_t thread, void **retval)` - ожидание завершения потока и синхронизация.
- `clock_gettime(clockid_t clk_id, struct timespec *tp)` - точный замер монотонного времени.
- `write(int fd, const void *buf, size_t count)` - низкоуровневый вывод данных в дескриптор терминала.

Результатом данной лабораторной работы стало вычисление ускорения и эффективности для различного числа потоков. При количестве потоков $p = 1$ величина ускорения составляет $S = 0.98$, что обусловлено системными накладными расходами операционной системы на вызовы функций `pthread_create` и `pthread_join`, отсутствующими в чистом последовательном алгоритме. Остальные данные совпадают с теоретическими значениями.

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
Последовательная	1502.62	1.00	1.00 (100.00%)
1	1525.77	0.98	0.98 (98.48%)
2	922.45	1.63	0.81 (81.45%)
4	588.07	2.56	0.64 (63.88%)
8	470.55	3.19	0.40 (39.92%)
12	254.73	5.90	0.49 (49.16%)
16	371.02	4.05	0.25 (25.31%)
128	379.65	3.96	0.03 (3.09%)
1024	414.01	3.63	0.004 (0.35%)

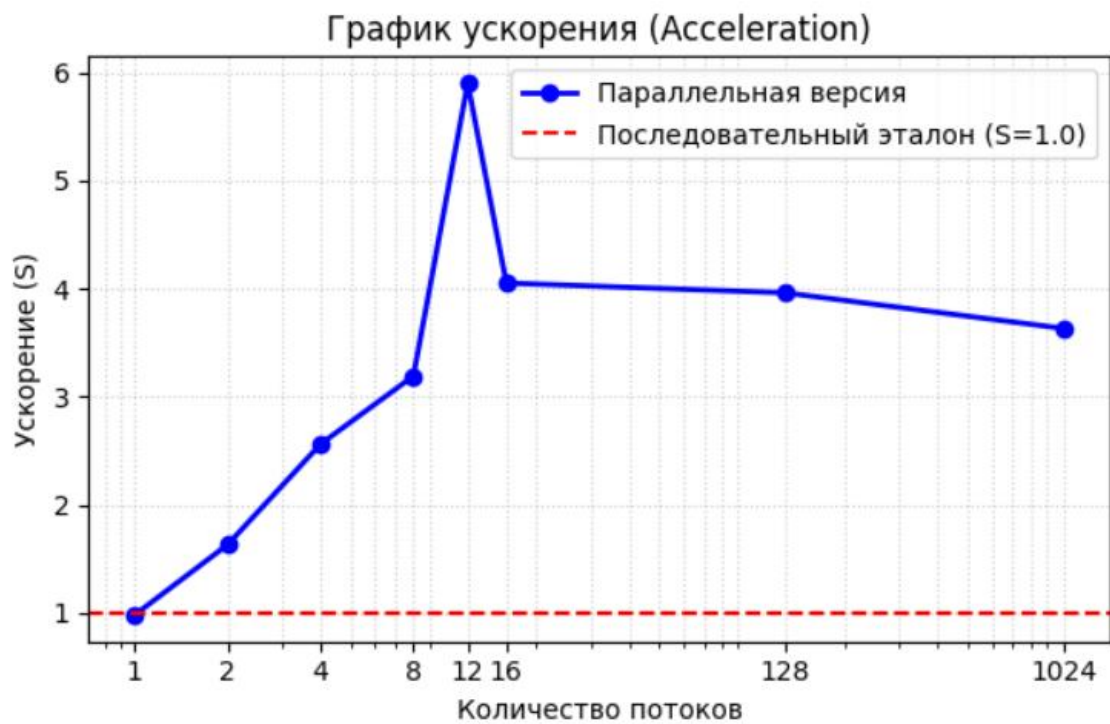


рис. 1, зависимость ускорения от количества потоков

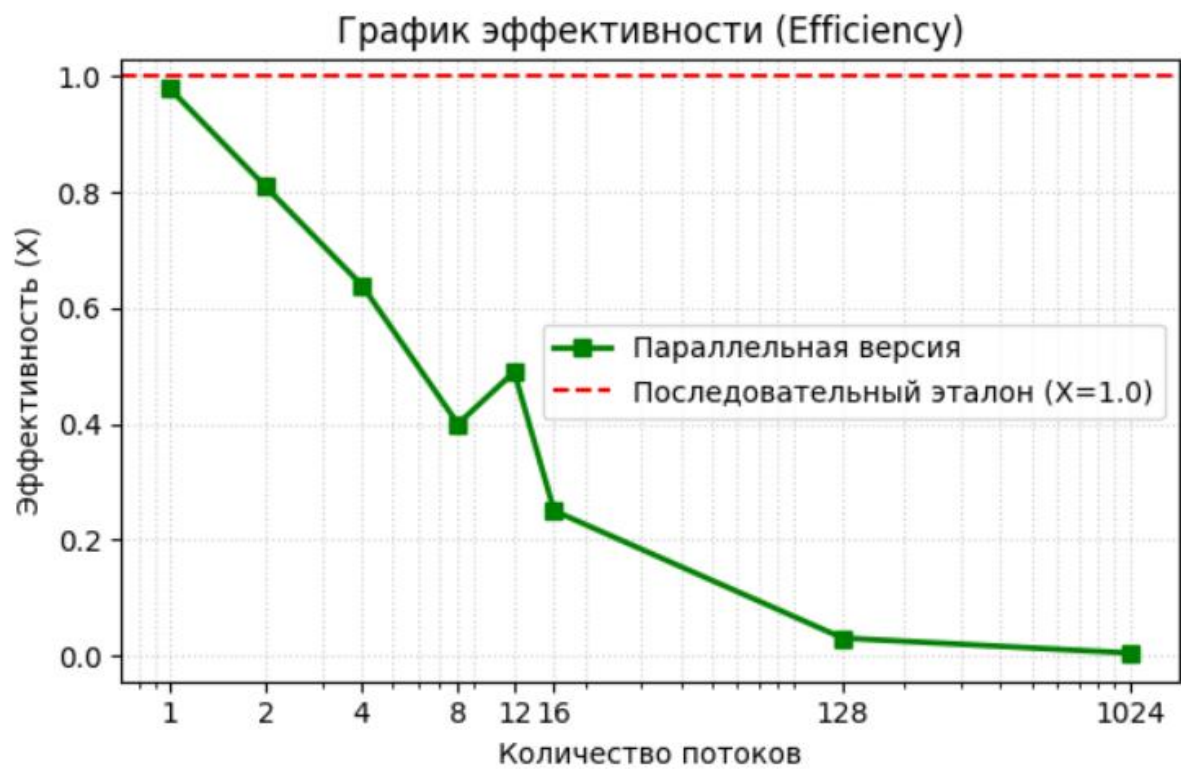


рис. 2, зависимость эффективности от количества потоков